

实验七：I/O 模型演进：从 C10K 到协程（参考手册）

1. 本实验研究的问题

网络服务器需要同时面对大量连接。一个连接可能长时间没有数据，也可能在任意时刻变为可读或可写。

本实验包含三个基础必做任务和一个可选协程任务，并在任务三和可选任务四之间分析多轮 Reactor：

多线程阻塞 I/O

→ 单线程非阻塞忙轮询

→ `epoll` 短连接事件通知

→ 分析 `Reactor` 多轮状态机的复杂度

→ `epoll` + 协程（可选）

前三个任务逐步暴露并解决前一个模型的问题；可选任务四不改变底层 I/O 等待机制，而是改善多轮业务的代码组织方式：

阶段	解决的问题	新暴露的问题
多线程阻塞 I/O	顺序代码容易编写	线程资源随连接数增长
非阻塞忙轮询	不再为每个连接创建线程	CPU 浪费在无效查询
<code>epoll</code> 短连接	只处理真正就绪的 fd	尚未处理跨多轮事件的业务状态
<code>Reactor</code> 多轮状态机（过渡分析）	支持多轮非阻塞业务	需要显式保存状态和跨事件数据
<code>epoll</code> + 协程（可选）	底层仍使用 <code>epoll</code> ，隐藏执行位置和局部状态保存	运行时设计更复杂

2. 阻塞、非阻塞、同步和异步

“阻塞/非阻塞”和“同步/异步”描述的是两个不同维度，不能把“非阻塞”直接等同于“异步”。

维度	关注的问题
阻塞 / 非阻塞	I/O 暂时无法完成时，当前调用是否等待
同步 / 异步	I/O 操作由应用程序主动完成，还是由系统完成后通知结果

2.1 阻塞与非阻塞

阻塞和非阻塞描述系统调用在暂时无法完成时如何返回。

对阻塞 `socket` 调用：

```
ssize_t n = read(fd, buf, size);
```

如果当前没有数据，调用线程会休眠，直到：

- 数据到达；
- 对端关闭；
- 信号中断；
- 发生错误。

非阻塞 socket 需要通过 `fcntl()` 修改文件状态标志。具体命令和标志见 4.5 节。没有数据时，`read()` 不等待，而是立即返回：

```
n == -1
errno == EAGAIN || errno == EWOULDBLOCK
```

`EAGAIN` 和 `EWOULDBLOCK` 不表示连接损坏，只表示当前时刻无法完成操作。程序应在条件变化后重试，而不是立即关闭连接。

非阻塞只表示单次调用不会等待，不代表程序一定高效。任务二不断扫描所有连接，即使每次 `read()` 都立即返回 `EAGAIN`，仍然会持续消耗 CPU。

2.2 同步与异步

同步和异步关注 I/O 操作由谁完成，以及应用程序得到的是“就绪通知”还是“完成结果”。

`epoll` 通知应用程序：

这个 `fd` 现在可读

应用程序收到通知后，仍然需要主动调用 `read()` 把数据读入用户缓冲区。因此在操作系统 I/O 模型的严格语境下，`epoll + read()` 仍属于同步 I/O。

完成型接口通知应用程序：

先前提交的 `recv` 操作已经完成，结果是 `N` 字节

`io_uring` 可以通过提交队列和完成队列表达这种完成型处理流程。

需要注意：具体数据传输是否使用 DMA、是否发生系统调用、是否启用轮询，取决于设备、内核和 `io_uring` 配置。不能简单把所有 `io_uring` 操作描述为“完全没有系统调用”或“所有数据都由 DMA 直接搬到用户缓冲区”。

2.3 本实验各阶段的归类

阶段	调用方式	I/O 完成方式
任务一：多线程阻塞 I/O	阻塞	同步，线程调用 <code>read()</code> 、 <code>write()</code>
任务二：忙轮询	非阻塞	同步，程序反复调用 <code>read()</code> 、 <code>write()</code>

阶段	调用方式	I/O 完成方式
任务三: epoll Reactor	非阻塞, <code>epoll_wait()</code> 可阻塞等待事件	同步, 程序在 fd 就绪后调用 <code>read()</code> 、 <code>write()</code>
可选任务四: epoll + 协程	底层仍是非阻塞 socket 和 <code>epoll</code>	同步, 协程只改变业务代码的组织与恢复方式

这里“任务三使用非阻塞 I/O”与“线程阻塞在 `epoll_wait()`”并不矛盾: 前者描述监听 socket 和客户端 socket 的操作方式, 后者描述事件循环在没有就绪事件时如何等待。

3. TCP 连接与服务器队列

3.1 `listen` 和 `accept`

服务器建立监听 socket 的典型流程:

```
socket()
→ setsockopt()
→ bind()
→ listen()
→ accept()
```

`listen()` 之后, 内核维护连接队列。TCP 握手完成不等于应用程序已经调用 `accept()`, 也不等于服务器已经为该连接创建工作线程。

因此客户端可能观察到:

```
connect() 成功
```

但服务端稍后因为 `pthread_create()` 失败而关闭连接。客户端必须继续发送请求并读取有效响应, 才能确认应用层处理成功。

3.2 `accept()` 后立即关闭

服务端已经取得客户端 fd 后立即 `close()`, 客户端可能看到:

- `read()` 返回 0;
- `ECONNRESET`;
- 空响应;
- HTTP 请求失败。

具体表现取决于关闭时是否还有未读取的数据以及 TCP 状态。

3.3 暂停 `accept`

应用程序暂时不调用 `accept()` 时，已完成握手的连接可能继续停留在内核队列中。队列满后，新连接可能经历重传、延迟或超时。

这和“先 `accept()` 再立即关闭”不同：

- 前者体现背压和排队；
- 后者表示应用程序主动接收后拒绝。

4. 基础 API 使用速查

本节汇总基础任务和可选任务四的 TODO 需要使用的主要 Linux 和 POSIX API。每个 API 给出函数原型、参数候选值和选择依据。学生需要根据任务要求判断参数组合，并完成具体调用。

4.1 创建 TCP 监听 socket

需要包含：

```
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
```

socket

```
int socket(int domain, int type, int protocol);
```

创建一个 socket，成功时返回非负文件描述符，失败时返回 `-1` 并设置 `errno`。

第 1 个参数 `domain` 的常见候选值：

候选值	含义
<code>AF_INET</code>	IPv4 网络通信
<code>AF_INET6</code>	IPv6 网络通信
<code>AF_UNIX</code>	同一主机上的进程间通信

第 2 个参数 `type` 的常见候选值：

候选值	含义
<code>SOCK_STREAM</code>	面向连接的可靠字节流；在 Internet 地址族中通常用于 TCP
<code>SOCK_DGRAM</code>	无连接的数据报；在 Internet 地址族中通常用于 UDP
<code>SOCK_RAW</code>	直接访问较低层协议

第 3 个参数 `protocol` 的常见候选值：

候选值	含义
0	根据地址族和 socket 类型选择默认协议
IPPROTO_TCP	明确指定 TCP
IPPROTO_UDP	明确指定 UDP

选择三个参数时分别判断：

- 1. domain：任务要求使用 IPv4、IPv6，还是本机进程间通信；
- 2. type：任务要求可靠的连接式字节流，还是无连接的数据报；
- 3. protocol：使用前两个参数对应的默认协议，还是显式指定具体协议。

对于“IPv4 TCP socket”：

- 第一个参数应从地址族候选中选择 IPv4；
- 第二个参数应从传输类型候选中选择可靠、面向连接的字节流；
- 第三个参数既可以选择默认协议，也可以显式指定 TCP。

因此，第三个参数存在两种合理写法。只要三个参数组合后确实创建 IPv4 TCP socket，都应视为正确。

bind

```
int bind(int fd, const struct sockaddr *addr, socklen_t addr_len);
```

参数含义：

位置	参数	应传入的内容
第 1 个参数	fd	需要绑定本地地址的 socket fd
第 2 个参数	addr	指向本地地址结构，并转换为 const struct sockaddr *
第 3 个参数	addr_len	第 2 个参数所指地址结构的实际大小

成功返回 0，失败返回 -1。常见错误包括端口已被占用或权限不足。

listen

```
int listen(int fd, int backlog);
```

参数含义：

位置	参数	应传入的内容
第 1 个参数	fd	已经完成地址绑定的 socket fd
第 2 个参数	backlog	内核连接队列长度的提示值

`backlog` 可以使用程序中定义的队列长度常量，也可以使用系统允许范围内的正整数。它不代表服务器能够创建同样数量的工作线程。

返回值：

- `0`：监听设置成功；
- `-1`：设置失败，并设置 `errno`。

`accept`

```
int accept(int listen_fd, struct sockaddr *peer, socklen_t *peer_len);
```

从监听队列取得一个连接，成功返回新的客户端 fd。

参数含义：

位置	参数	应传入的内容
第 1 个参数	<code>listen_fd</code>	已进入监听状态的 socket fd
第 2 个参数	<code>peer</code>	接收客户端地址的结构体指针；不需要地址时传空指针
第 3 个参数	<code>peer_len</code>	指向地址结构长度变量；第 2 个参数为空时也传空指针

第 2、3 个参数必须配套选择：

<code>peer</code>	<code>peer_len</code>	适用情况
空指针	空指针	不需要取得客户端地址
客户端地址结构指针	地址长度变量指针	需要取得客户端 IP 和端口

监听 fd 保持不变，可以继续接收其他连接。阻塞监听 fd 在没有连接时等待；非阻塞监听 fd 返回 `-1`，并通常设置：

```
errno == EAGAIN || errno == EWOULDBLOCK
```

4.2 读写与关闭连接

需要包含：

```
#include <unistd.h>
```

`read`

```
ssize_t read(int fd, void *buf, size_t count);
```

返回值含义：

- 大于 `0`：实际读取的字节数；

- 等于 0：对端已经关闭发送方向；
- 等于 -1：发生错误，具体原因查看 `errno`。

参数含义：

位置	参数	应传入的内容
第 1 个参数	<code>fd</code>	当前需要读取的连接 <code>fd</code>
第 2 个参数	<code>buf</code>	接收数据的缓冲区起始地址
第 3 个参数	<code>count</code>	本次最多写入缓冲区的字节数

如果后续需要在数据末尾写入 `'\0'`，`count` 应小于缓冲区总容量，并只在返回值大于 0 时写入终止符。

write

```
ssize_t write(int fd, const void *buf, size_t count);
```

成功时返回本次实际发送的字节数。返回值可能小于 `count`，复杂协议需要保存发送位置并继续发送。

参数含义：

位置	参数	应传入的内容
第 1 个参数	<code>fd</code>	需要发送数据的连接 <code>fd</code>
第 2 个参数	<code>buf</code>	待发送数据的起始地址
第 3 个参数	<code>count</code>	希望发送的字节数

close

```
int close(int fd);
```

第 1 个参数 `fd` 是需要释放的文件描述符。成功返回 0；失败返回 -1 并设置 `errno`。连接处理结束、线程创建失败、事件注册失败等路径都需要关闭不再使用的 `fd`。

4.3 TODO 使用的 C 标准库函数

需要包含：

```
#include <stdlib.h>
#include <string.h>
```

free

```
void free(void *ptr);
```

第 1 个参数 `ptr` 指向需要释放的动态内存；也可以传空指针。传给工作线程的参数在取出所需数据后应由明确的一方释放，避免泄漏或重复释放。

strlen

```
size_t strlen(const char *text);
```

第 1 个参数 `text` 指向以 `'\0'` 结尾的字符串。函数返回字符串结束符之前的字符数。向连接写入固定字符串时，可以用它确定希望发送的字节数。

4.4 POSIX 线程 API

需要包含：

```
#include <pthread.h>
```

线程入口函数类型为：

```
void *worker(void *arg);
```

创建线程：

```
int pthread_create(pthread_t *thread,  
                   const pthread_attr_t *attr,  
                   void *(*start_routine)(void *),  
                   void *arg);
```

参数含义：

位置	参数	选择依据
第 1 个参数	<code>thread</code>	指向用于接收线程标识的 <code>pthread_t</code> 变量
第 2 个参数	<code>attr</code>	使用已初始化的线程属性，或传空指针采用默认属性
第 3 个参数	<code>start_routine</code>	新线程需要执行的入口函数
第 4 个参数	<code>arg</code>	传给入口函数的参数指针；没有参数时可传空指针

POSIX 线程函数通常直接返回错误码，不依赖 `errno`：

```
if (ret != 0)  
    fprintf(stderr, "%s\n", strerror(ret));
```

学生框架已经准备好线程属性。调用 `pthread_create()` 时需要使用框架中的属性对象、线程入口函数和客户端参数。传给线程的参数必须在线程取得之前保持有效。

4.5 使用 `fcntl` 设置非阻塞

需要包含：

```
#include <fcntl.h>
```

函数原型：

```
int fcntl(int fd, int command, ...);
```

`fcntl()` 用于读取或修改一个文件描述符的属性。第三个参数是否存在以及类型是什么，由第二个参数 `command` 决定。

参数含义：

位置	参数	含义与选择依据
第 1 个参数	<code>fd</code>	需要设置为非阻塞的监听 <code>fd</code> 或客户端 <code>fd</code>
第 2 个参数	<code>command</code>	判断本次需要读取当前标志，还是写入新标志
第 3 个参数（可变参数）	附加值	读取类命令通常传 <code>0</code> ；设置类命令传入新标志

第 2 个参数 `command` 的常见候选值：

第 2 个参数候选值	作用	第 3 个参数（可变参数）
<code>F_GETFL</code>	读取文件状态标志	通常为 <code>0</code>
<code>F_SETFL</code>	设置文件状态标志	新的标志组合
<code>F_GETFD</code>	读取文件描述符标志	通常为 <code>0</code>
<code>F_SETFD</code>	设置文件描述符标志	新的描述符标志

文件状态标志和文件描述符标志不是同一类数据：

标志类别	第 3 个参数中可使用的候选值	适用的第 2 个参数 <code>command</code>	作用
文件状态标志	<code>O_NONBLOCK</code>	<code>F_SETFL</code>	系统调用暂时无法完成时立即返回
文件状态标志	<code>O_APPEND</code>	<code>F_SETFL</code>	每次写入都定位到文件末尾
文件描述符标志	<code>FD_CLOEXEC</code>	<code>F_SETFD</code>	执行新程序时关闭该 <code>fd</code>

把 socket 设置为非阻塞需要分为两个步骤：

1. 读取该 fd 当前的**文件状态标志**；
2. 保留原有标志，加入非阻塞标志后写回。

选择命令时应注意：

- 需要读取或设置 `O_NONBLOCK` 时，应选择操作“文件状态标志”的命令；
- `F_GETFD`、`F_SETFD` 操作的是 `FD_CLOEXEC` 等描述符标志，不能用于设置 `O_NONBLOCK`；
- 写回新标志时不能只传入 `O_NONBLOCK`，否则可能清除 fd 原有的其他状态标志。

返回值：

- 读取标志成功：返回当前标志值；
- 设置标志成功：返回 `0`；
- 失败：返回 `-1` 并设置 `errno`。

因此应分别检查两次调用的返回值。失败时可以使用 `perror()` 输出错误，并根据任务要求结束程序或清理当前连接。

socket 设置为非阻塞后：

- 没有新连接时，`accept()` 可能返回 `-1`，并设置 `EAGAIN` 或 `EWOULDBLOCK`；
- 没有数据时，`read()` 可能返回 `-1`，并设置 `EAGAIN` 或 `EWOULDBLOCK`；
- 暂时无法继续发送时，`write()` 也可能出现相同结果。

这些结果表示“当前不能完成”，不表示连接已经损坏。

4.6 epoll API

需要包含：

```
#include <sys/epoll.h>
```

`epoll_create1`

```
int epoll_create1(int flags);
```

第 1 个参数 `flags` 的常见候选值：

候选值	含义
<code>0</code>	不附加创建标志
<code>EPOLL_CLOEXEC</code>	进程执行新程序时自动关闭该 fd

成功返回 `epoll fd`，失败返回 `-1`。使用完后需要 `close(epoll_fd)`。

epoll_event

`struct epoll_event` 是传给 `epoll_ctl()`、由 `epoll_wait()` 返回的事件结构。主要包含：

结构体成员	填写内容
<code>ev.events</code>	关注的事件位掩码，可以使用按位或组合多个事件标志
<code>ev.data</code>	与该事件关联、供应用程序取回的数据

成员 `ev.events` 的常见候选值：

候选值	适用条件
<code>EPOLLIN</code>	需要等待可读、对端关闭或新连接到达
<code>EPOLLOUT</code>	有尚未发送完的数据，需要等待可写
<code>EPOLLET</code>	使用边沿触发方式
<code>EPOLLONESHOT</code>	一次通知后暂时禁用该 fd

成员 `ev.data` 是联合体，常用成员如下：

联合体成员	保存内容	适用情况
<code>ev.data.fd</code>	文件描述符	事件返回后直接根据 fd 分支处理
<code>ev.data.u32</code>	32 位无符号整数	保存较小的编号或标签
<code>ev.data.u64</code>	64 位无符号整数	保存协程 ID、标签或编码后的数据
<code>ev.data.ptr</code>	指针	保存连接对象或请求上下文地址

这些联合体成员共享同一块存储空间。注册时写入哪个成员，处理 `epoll_wait()` 返回事件时就应该读取同一个成员。

epoll_ctl

```
int epoll_ctl(int epoll_fd, int operation,
              int fd, struct epoll_event *event);
```

参数含义：

位置	参数	应传入的内容
第 1 个参数	<code>epoll_fd</code>	<code>epoll_create1()</code> 返回的 epoll 实例 fd
第 2 个参数	<code>operation</code>	本次执行新增、修改还是删除
第 3 个参数	<code>fd</code>	被注册、修改或删除的目标 fd
第 4 个参数	<code>event</code>	指向描述关注事件和用户数据的 <code>struct epoll_event</code>

第 2 个参数 `operation` 的候选值:

第 2 个参数 <code>operation</code> 的候选值	作用	第 4 个参数 <code>event</code>
<code>EPOLL_CTL_ADD</code>	注册一个尚未加入的 fd	指向事件结构
<code>EPOLL_CTL_MOD</code>	修改已注册 fd 的事件	指向新的事件结构
<code>EPOLL_CTL_DEL</code>	删除已注册 fd	Linux 中可传空指针

学生应根据 fd 当前是否已经注册，以及本次操作是新增、修改还是删除来选择。成功返回 `0`，失败返回 `-1`。

`epoll_wait`

```
int epoll_wait(int epoll_fd, struct epoll_event *events,
               int max_events, int timeout_ms);
```

参数含义:

位置	参数	应传入的内容
第 1 个参数	<code>epoll_fd</code>	要等待的 epoll 实例 fd
第 2 个参数	<code>events</code>	用于接收就绪事件的结构体数组首地址
第 3 个参数	<code>max_events</code>	第 2 个参数数组最多可容纳的事件数量
第 4 个参数	<code>timeout_ms</code>	本次等待的超时时间，单位为毫秒

- 返回正数：本次得到的就绪事件数量；
- 返回 `0`：等待超时；
- 返回 `-1`：发生错误；

第 4 个参数 `timeout_ms` 的常见候选值:

值	行为
<code>-1</code>	一直等待，直到有事件或被信号中断
<code>0</code>	立即返回，不等待
正整数	最多等待指定毫秒数

只应遍历 `events[0]` 到 `events[n - 1]`，它们就是本轮需要处理的就绪对象。

4.7 可选任务四中的协程函数

可选任务四使用的协程创建、切换和调度函数已经在框架中实现。TODO 需要调用的函数包括：

```
static int coroutine_get_current_id(void);
static void coroutine_yield(void);
static ssize_t co_read_line(int fd, char *buf, size_t size);
```

函数	参数与返回值
<code>coroutine_get_current_id()</code>	无参数；返回当前正在运行的协程 ID
<code>coroutine_yield()</code>	无参数、无返回值；挂起当前协程并返回调度器
<code>co_read_line()</code>	第 1 个参数 <code>fd</code> 是客户端 fd；第 2 个参数 <code>buf</code> 是接收缓冲区；第 3 个参数 <code>size</code> 是缓冲区容量；返回读取结果

学生需要根据 `co_read()` 的控制流程决定这些函数的调用位置，并在业务函数中检查 `co_read_line()` 的返回值。

4.8 错误处理规则

`perror`

```
void perror(const char *prefix);
```

`perror()` 输出两部分内容：

```
prefix: errno 对应的错误说明
```

第 1 个参数 `prefix` 应选择能够指出失败操作的文字。候选方式包括：

写法	适用情况
API 名称，如 <code>"socket"</code>	能直接定位失败的系统调用
API 名称和用途，如 <code>"epoll_ctl listen"</code>	同一个 API 在多处使用，需要区分操作对象
空字符串或空指针	只输出系统错误，不利于定位问题

Linux 系统调用通常在失败时返回 `-1` 并设置 `errno`。这类调用可以在检查返回值后使用 `perror()`，例如 `socket()`、`bind()`、`listen()`、`accept()`、`read()`、`write()`、`fcntl()` 和 `epoll` 系列函数。

`exit`

```
void exit(int status);
```

`exit()` 结束整个进程。第 1 个参数 `status` 的常见候选值：

候选值	含义
<code>EXIT_SUCCESS</code> 或 <code>0</code>	程序正常结束
<code>EXIT_FAILURE</code>	程序因失败结束

候选值	含义
非零整数	表示失败，具体含义由程序约定

选择是否调用 `exit()` 时，应判断错误影响范围：

- 监听 socket 创建、绑定或监听失败：服务器无法继续提供服务，通常结束进程；
- 单个客户端连接失败：只清理该连接，服务器继续运行；
- 临时错误，如 `EINTR` 或非阻塞操作的 `EAGAIN`：根据错误语义重试或等待，不结束进程。

需要使用 `perror()` 时包含：

```
#include <stdio.h>
```

需要使用 `exit()`、`EXIT_SUCCESS` 或 `EXIT_FAILURE` 时包含：

```
#include <stdlib.h>
```

POSIX 线程函数的错误

`pthread_create()` 等 POSIX 线程函数通常直接返回错误码，不保证设置 `errno`，因此不应直接使用 `perror()`。可以使用：

```
strerror(error_code)
```

把返回的错误码转换为说明文字，再通过 `fprintf()` 输出。

以下错误通常表示可以重试，而不是立即终止整个服务器：

- `EINTR`：调用被信号中断；
- `EAGAIN`、`EWOULDBLOCK`：非阻塞操作当前无法完成；
- `ECONNABORTED`：等待队列中的某个连接在 `accept()` 前中止。

资源已经分配后发生错误时，还应释放相应的 fd、内存、线程属性或 `epoll` 实例。

5. io_uring 进阶知识

5.1 SQ 和 CQ

`io_uring` 使用两类环形队列：

- SQ: Submission Queue，提交请求；
- CQ: Completion Queue，取得完成结果。

常见流程：

```
io_uring_get_sqe
→ io_uring_prep_*
→ 设置 user_data
→ io_uring_submit
→ io_uring_wait_cqe
→ 读取 cqe->res
→ io_uring_cqe_seen
```

5.2 cqe->res

完成结果直接放在：

```
cqe->res
```

- 非负数：操作结果，例如收发字节数或新客户端 fd；
- 负数：负的 errno，例如 `-ECONNRESET`。

不能只调用 `perror()`，因为错误码未必存放在全局 `errno` 中：

```
fprintf(stderr, "%s\n", strerror(-cqe->res));
```

5.3 user_data 与请求生命周期

多个请求同时在内核中执行，CQE 返回顺序不一定等于提交顺序。程序需要通过 `user_data` 找回：

- 操作类型；
- 所属连接；
- 缓冲区；
- 当前偏移；
- 后续应提交的操作。

请求上下文至少持续到 CQE 被处理。过早释放会造成悬空指针。

建议把 `user_data` 指向一个小的请求对象，而不是只保存 fd。因为同一个连接可能先后提交 `recv username`、`send prompt`、`recv password`、`send result`，CQE 回来时程序必须知道“这是哪一种操作完成了”。

可以采用如下组织方式：

```
typedef enum {
    OP_ACCEPT,
    OP_RECV_USERNAME,
    OP_SEND_PROMPT,
    OP_RECV_PASSWORD,
    OP_SEND_RESULT
} OpType;

typedef struct Conn Conn;

typedef struct Request {
```

```
OpType op;  
Conn *conn;  
} Request;
```

`accept` 操作没有现成连接，可以让 `Request.conn == NULL`。处理 `accept` 完成后，再创建新的 `Conn`，并提交该连接的第一次 `recv`。

连接对象中至少需要保存：

```
struct Conn {  
    int fd;  
    char username[64];  
    size_t username_len;  
    char password[64];  
    size_t password_len;  
    char send_buf[256];  
    size_t send_len;  
    size_t send_pos;  
};
```

如果一次 `recv` 没有读到换行符，不要丢弃已经读到的内容。应把新字节追加到 `username` 或 `password` 缓冲区中，然后继续提交同一阶段的 `recv`。

5.4 完成链

两轮登录可以表示为：

```
accept 完成  
→ 提交 recv username  
→ recv 完成  
→ 提交 send prompt  
→ send 完成  
→ 提交 recv password  
→ recv 完成  
→ 提交 send result  
→ send 完成  
→ close
```

每个完成事件决定下一次提交什么操作。

完成链的核心不是“在一个函数里阻塞等待下一步”，而是“处理一个 CQE 后，提交下一批 SQE”。可以按下面的逻辑设计：

OP_ACCEPT 完成：

如果成功，创建连接对象，提交 `recv username`
无论是否成功，都继续提交下一次 `accept`

OP_RECV_USERNAME 完成：

如果读到完整一行，准备 `password` 提示，提交 `send prompt`
如果还没读到完整一行，继续提交 `recv username`

OP_SEND_PROMPT 完成：

如果还有剩余字节，继续提交 `send prompt`
如果提示发送完，提交 `recv password`

OP_RECV_PASSWORD 完成：

如果读到完整一行，准备登录结果，提交 `send result`
如果还没读到完整一行，继续提交 `recv password`

OP_SEND_RESULT 完成：

如果还有剩余字节，继续提交 `send result`
如果结果发送完，关闭 `fd` 并释放连接对象

每个“提交操作”的函数都应该做三件事：

1. 从 ring 中取得 SQE；
2. 使用 `io_uring_prep_*` 填写操作；
3. 创建或复用请求对象，并通过 `io_uring_sqe_set_data()` 绑定到 SQE。

例如提交一次接收可以抽象为：

```
submit_recv(conn, OP_RECV_USERNAME)
```

它内部准备 `io_uring_prep_recv()`，并把 `Request{OP_RECV_USERNAME, conn}` 放入 `user_data`。真正的代码可以自行决定 `Request` 是动态分配、连接内嵌，还是使用对象池；关键是 CQE 到达之前不能释放。

5.5 部分发送

`send` 完成并不保证发送了全部数据：

```
send_pos += cqe->res;
```

如果：

```
send_pos < send_len
```

应继续提交剩余部分。

部分接收也要认真处理。TCP 是字节流，不能假设一次 `recv` 正好读到一行用户名或一行密码。一次 `recv` 可能出现：

- 只读到半行；
- 一次读到完整一行；
- 一次读到一行后还带着后续数据；
- 返回 0，表示对端关闭；
- 返回负数错误。

进阶任务可以采用简化协议：用户名和密码各占一行，每行不超过固定长度。读到换行符后，将换行符前的内容作为本轮输入。如果缓冲区已满仍未读到换行符，可以返回错误响应或直接关闭连接。

为了降低实现难度，可以让每一轮 `recv` 只处理当前阶段需要的一行。若一次 `recv` 中读到了额外字节，进阶实现可以先将额外字节保存到连接缓冲区；简化实现也可以在报告中说明协议假设为“客户端按提示逐行发送”，并在测试中按该假设输入。

发送提示和结果时，应把待发送内容统一放入 `send_buf`，通过 `send_pos` 记录已经完成的字节数：

```
send_buf = "password: "  
send_len = strlen(send_buf)  
send_pos = 0
```

每次 CQE 返回正数后更新 `send_pos`。只有 `send_pos == send_len` 时，才进入下一阶段。

5.6 环境限制

`io_uring` 可能受以下因素影响：

- Linux 内核版本；
- 容器 `seccomp`；
- 虚拟化平台；
- 系统安全策略；
- `liburing` 版本。

初始化失败时应记录实际错误码，再判断是环境问题还是代码问题。

常见排查方向：

- 编译阶段找不到 `liburing.h`：检查是否安装 `liburing-dev`；
- 链接阶段出现 `undefined reference`：检查是否加上 `-luring`；
- `io_uring_queue_init()` 返回负数：打印 `strerror(-ret)`；
- 在容器或受限 WSL 环境中初始化失败：检查内核版本、`seccomp` 和运行权限；
- CQE 的 `res` 为负数：打印 `strerror(-cqe->res)`，不要只看 `errno`。

建议在程序启动时打印：

```
io_uring server start, port=4000
```

在关键完成事件打印简短日志：

```
accept fd=...  
recv username fd=... n=...  
send prompt fd=... done=.../...  
recv password fd=... n=...  
close fd=...
```

日志不宜过多，否则大量并发连接测试时会严重影响性能。调通功能后，可以只保留错误日志和统计信息。

5.7 和 epoll Reactor 的关键差异

epoll 的事件含义是“fd 现在可能可读或可写”。程序收到事件后，还需要自己调用 read() 或 write()，并处理 EAGAIN。

io_uring 的 CQE 含义是“之前提交的某个 I/O 操作已经完成”。程序处理 CQE 时，直接读取完成结果 cqe->res，再决定提交下一步操作。

对比：

维度	epoll Reactor	io_uring Proactor
等待对象	fd 就绪事件	I/O 操作完成事件
事件到达后	应用程序调用 read/write	应用程序读取 CQE 结果
上下文绑定	通常由 fd 查连接对象	通常由 user_data 查请求对象
下一步推进	事件处理函数修改状态并注册事件	CQE 处理函数提交新的 SQE

进阶报告中不要把 io_uring 简单写成“更快的 epoll”。更准确的说法是：它让程序围绕“提交操作”和“处理完成”组织代码，和 Reactor 围绕“等待 fd 就绪”组织代码不同。

5.8 建议实现顺序

不要一开始就写完整两轮登录。可以按下面顺序推进：

1. 只实现 io_uring_queue_init() 和固定监听 socket；
2. 提交一个 accept，确认客户端连接能产生 CQE；
3. 每次 accept 完成后立即提交下一次 accept；
4. 对新连接提交一次 recv，收到任意数据后直接回复固定字符串；
5. 加入部分发送处理；
6. 加入按换行读取用户名；
7. 加入发送密码提示；
8. 加入按换行读取密码；
9. 加入登录结果和连接释放；
10. 最后再做大量空闲连接和交错输入测试。

这样可以把问题定位在当前新增的一步，避免同时调试 accept、recv、send、状态切换和资源释放。

6. 多 Reactor 进阶知识

6.1 基本结构

多 Reactor 服务器通常分为：

主线程：

`accept` → 选择工作线程 → 传递 `client fd`

工作线程：

独立 `epoll_wait` → 处理所属连接

主线程不处理客户端业务。每个连接一旦分配给某个工作线程，其状态和 `fd` 都由该线程独占。

推荐的对象划分如下：

```
typedef struct Conn {
    int fd;
    int state;
    char recv_buf[256];
    size_t recv_used;
    char send_buf[256];
    size_t send_len;
    size_t send_pos;
} Conn;

typedef struct worker {
    int id;
    pthread_t tid;
    int epfd;
    int notify_fd;
    pthread_mutex_t lock;
    int pending_fds[1024];
    int pending_head;
    int pending_tail;
    unsigned long accepted_count;
    unsigned long completed_count;
} worker;
```

实际代码不必完全照抄这个结构，但需要体现三个边界：

- 主线程负责监听 `fd` 和新连接分配；
- 工作线程负责自己的 `epoll_wait()`；
- 连接状态只由所属工作线程读写。

6.2 为什么每个工作线程需要独立 `epoll`

独立 `epoll` 可以让每个工作线程：

- 只等待自己负责的连接；
- 在本线程中修改连接状态；
- 避免多个线程同时处理同一事件；
- 减少共享状态。

6.3 跨线程传递 fd

文件描述符是进程级资源，同一进程中的线程都能使用它。但主线程仍需要把“有一个新 fd 待注册”通知工作线程。

典型结构：

```
互斥锁保护的待接管队列
+ eventfd 唤醒
```

主线程：

```
加锁
→ fd 入队
→ 解锁
→ write(eventfd)
```

工作线程：

```
epoll_wait 返回 eventfd
→ read(eventfd)
→ 取走队列
→ 把 fd 注册到自己的 epoll
```

这里的重点是：主线程不要直接把新客户端 fd 注册到工作线程的 epoll 实例中。虽然技术上可以拿到 `epfd` 调用 `epoll_ctl`，但这样会让“谁拥有这个连接”的边界变得混乱。更清晰的做法是：

1. 主线程 `accept()` 得到 `client_fd`；
2. 主线程选择目标工作线程；
3. 主线程把 `client_fd` 放入该工作线程的待接管队列；
4. 主线程写 `eventfd` 唤醒该工作线程；
5. 工作线程醒来后，从自己的队列取 fd；
6. 工作线程设置非阻塞、创建连接对象、注册到自己的 epoll。

如果主线程在 `accept()` 后已经设置了非阻塞，也可以把这一步放在主线程完成。无论放在哪里，都要保证每个 fd 只执行一次初始化和一次注册。

6.4 eventfd

创建：

```
int notify_fd = eventfd(0, EFD_NONBLOCK | EFD_CLOEXEC);
```

写入和读取的类型是 64 位无符号整数：

```
uint64_t one = 1;
write(notify_fd, &one, sizeof(one));
```

`eventfd` 适合用作事件循环中的跨线程唤醒源。

`eventfd` 读写时需要处理非阻塞语义。工作线程收到 `eventfd` 可读事件后，可以循环读取直到 `EAGAIN`，或者读取一次后直接清空队列。由于 `eventfd` 计数值可能合并多次唤醒，程序不能假设一次 `read(eventfd)` 对应一个 fd。正确做法是：

```
eventfd 可读
→ 读取并清空通知计数
→ 加锁
→ 取出当前队列中的所有 fd
→ 解锁
→ 逐个注册到本线程 epoll
```

队列也要考虑满队列情况。简单实现可以给每个工作线程设置足够大的固定数组，并在队列满时关闭新 fd，同时打印错误；更完整的实现可以使用动态队列。

6.5 连接所有权

必须建立清晰规则：

```
主线程在分配前拥有 fd
→ 入队后交给目标工作线程
→ 工作线程注册后负责状态、读写和关闭
```

如果两个线程同时：

- 修改同一个连接状态；
- 调用 `epoll_ctl`；
- 读写或关闭同一个 fd；

就可能出现数据竞争、重复关闭和 fd 复用错误。

fd 复用错误尤其隐蔽。例如线程 A 关闭了 fd 15，内核随后把新的客户端也分配为 fd 15；如果线程 B 手里还保存着旧 fd 15 并继续操作，就可能误伤新连接。避免这类问题的根本办法是：

- 一个连接只属于一个工作线程；
- 关闭前先从本线程 `epoll` 删除；
- 关闭后立即释放连接对象；
- 不把连接对象指针交给其他线程长期保存；
- 主线程入队后不再读写该 fd。

工作线程处理连接时，可以继续使用任务三中的状态机：

```
WAIT_USERNAME
→ SEND_PASSWORD_PROMPT
→ WAIT_PASSWORD
→ SEND_LOGIN_RESULT
→ CLOSE
```

读写仍然要处理 `EAGAIN`、对端关闭、部分发送和缓冲区满。多 Reactor 只是把连接分散到多个事件循环，并没有消除单个连接内部的非阻塞状态机。

6.6 工作线程数量

工作线程数量不是越多越好。数量增加可能带来：

- 更高并行度；
- 更多上下文切换；
- 更多 `epoll` 实例；
- 更复杂的负载不均；
- 更多缓存失效。

测试 1、2、4 个工作线程时，应同时记录：

- 各线程接管连接数；
- 请求完成数；
- CPU 时间；
- 总耗时；
- 是否出现明显负载偏斜。

如果 4 个工作线程中某一个接管了明显更多连接，常见原因包括：

- round-robin 计数写错；
- 某些 fd 入队失败后未统计；
- 工作线程队列满导致连接被关闭；
- 测试连接数太少，看不出均衡；
- 日志输出过多，让某些线程被 I/O 拖慢。

进阶报告分析时，不要求 1、2、4 个工作线程一定呈现线性加速。对于这个实验的两轮登录协议，瓶颈可能在客户端输入节奏、日志输出、单机调度或连接保持时间。报告重点应放在：

- 多个工作 Reactor 是否确实各自拥有独立 `epoll`；
- 连接是否被稳定分配到不同工作线程；
- 大量空闲连接是否不会导致忙轮询；
- 增加线程后是否带来额外调度和同步成本。

6.7 建议实现顺序

多 Reactor 也不要一次写完。推荐顺序：

1. 单线程 Reactor 先实现两轮登录状态机；
2. 抽出 `Conn` 结构和连接处理函数；
3. 创建工作线程，每个线程只打印启动信息；
4. 给每个工作线程创建独立 `epoll` 和 `eventfd`；
5. 主线程 `accept()` 后 round-robin 放入目标队列；
6. 工作线程收到 `eventfd` 后注册新 fd；
7. 工作线程先回复固定字符串，验证跨线程 fd 传递；

8. 再接入两轮登录状态机；

9. 最后加入统计信息和 1、2、4 线程对比测试。

每完成一步都应先用少量客户端测试，再扩大到大量等待连接。这样可以区分是线程分发问题、epoll 注册问题，还是业务状态机问题。

6.8 常见错误

多 Reactor 进阶任务常见错误包括：

- 忘记把 `eventfd` 加入工作线程 epoll，导致工作线程永远不知道新 fd 到达；
- 主线程入队后没有写 `eventfd`；
- `eventfd` 可读后只取一个 fd，队列中剩余 fd 长时间无人处理；
- 多个线程同时调用同一个连接的 `read()` 或 `write()`；
- 关闭 fd 后没有从 epoll 删除；
- 连接对象释放后，epoll 事件中仍保存旧指针；
- 所有工作线程共享一个 epoll fd，实际没有形成多 Reactor；
- 每个连接又创建一个线程，偏离进阶任务要求。

调试时可以临时打印：

```
main accept fd=17 -> worker 2
worker 2 take fd=17
worker 2 register fd=17
worker 2 close fd=17
```

当分发逻辑稳定后，再减少日志并做并发测试。